ShirohaChat 项目文档

项目小组 10 小组

小组成员 王俊民、张哲凯，任纹萱，陈瀚钦、封柏霖、熊俊啟

联系方式 15082342044

重庆师范大学软件工程系

第1章 摘要

在人类沟通长期依赖书信等传统方式的阶段,信息传递的延迟与成本长期存在:跨地域沟通困难、往返周期长,在“等待回信”的过程中,关键信息往往滞后甚至错过最佳时点。手机普及与移动互联网兴起后,人们对“随时联系、即时回应”的需求迅速成为日常生活的刚需,即时通讯由此从工具性产品演进为社会连接的基础设施,以更低成本、更高频率支撑人与人之间更及时、更亲密的沟通。

在这一背景下,本项目拟开发一款面向大众的普适即时通讯工具——ShirohaChat(桌面端)。产品聚焦日常沟通场景,服务朋友、家人、异地恋与同学等社交关系维护需求,围绕“及时触达、顺畅表达、稳定可靠、隐私可控”四个方向构建核心体验,覆盖私聊/群聊、文本/图片/语音/表情与文件发送、离线消息与重连、已读未读与消息搜索等基础能力,满足大众用户对即时联系与低成本连接的长期需求。

在开发过程中,团队基于面向对象分析与设计方法(OOAD)和统一建模语言(UML)进行系统建模,采用客户端-服务器架构(C/S)实现消息实时传递。项目引入非功能性需求(NFR)管理、协议契约设计、风险管理矩阵与量化性能指标。系统实现端到端延迟 ≤ 200ms、消息送达率 ≥ 99.99%、单节点支持 ≥ 5000 并发连接,具备断线重连与离线补发机制,确保消息零丢失。

关键词:大众即时通讯,社交沟通,私聊,群聊,离线消息,消息可靠性,低延迟,NFR

第2章　目录

摘要.....2

第 1 章 立项.....4

1.1. 项目起源与提案.....4

1.2. Business Case.....4

第 2 章 愿景.....5

2.1. 问题陈述.....5

1. 问题一.....5

2.2. 涉众与用户.....5

1. 涉众.....5

2. 用户.....5

2.3. 关键涉众和用户的需要.....5

2.4. 产品概述.....5

1. 产品定位陈述.....5

2. 完整的产品概述.....6

2.5. 产品特性.....6

2.6. 其他产品需求.....6

第 3 章 用况建模.....7

3.1. 术语表.....7

3.2. 想书的主要用况.....8

3.3. 用况的描述.....8

1. 简要描述.....8

2. 用况图.....8

3. 前置条件：.....8

4. 基本流：.....8

5. 子流.....11

5.1. 登录.....11

6. 备选流.....11

第 4 章 需求分析.....12

4.1. 健壮性分析.....12
4.2. 交互建模.....12
第 5 章 架构设计.....13
第 6 章 详细设计.....14
后记.....15
参考文献.....1U+0036

- 立项

- 1.1 项目起源与提案
  - 项目背景

在书信等传统沟通方式主导的年代,信息传递延迟高、成本高,跨地域沟通尤为困难,人与人之间的联系往往被时间与距离切割。手机普及后,人们对“随时联系、即时回应”的需求迅速增长,即时通讯成为日常生活的基础设施:它不仅承担信息传递,也承载关系维护、情感表达与社交连接。

在大众沟通场景中,用户对即时通讯的核心期待并不复杂,但要求极高:要快、要稳、要省心、要可信。若基础体验存在短板,会直接放大沟通成本与情绪摩擦,主要体现在三类问题:

1. 聊天入口多，重要信息难定位

   私聊与群聊同时进行,对话入口繁杂

   约定时间、地址、转账说明等关键信息容易被刷屏淹没

   回看成本高,错过信息后需要二次确认

2. 表达与互动能力需要完整覆盖

   大众沟通不仅是“发文字”,还包括语音、图片、表情与文件等多模态表达,并需要引用、转发、撤回等高频互动能力,以降低误解与重复沟通。

3. 弱网与离线场景下的可靠性与隐私诉求突出

   在网络波动、断线重连、设备切换等场景下,用户更关注“消息是否到达、是否重复、是否丢失”;同时,对隐私设置、黑名单与骚扰拦截也有明确预期。

  - 项目定位

"一款面向大众的普适桌面端即时通讯工具,聚焦朋友与家人等日常社交关系维护,以低延迟、高可靠与隐私可控为基础,提供私聊/群聊与多媒体表达能力,满足随时联系、即时回应的沟通刚需。"

  - 核心价值主张

更及时:降低“联系到对方”的等待成本,让沟通回到即时与自然的节奏

更亲密:以语音、图片、表情等表达方式承载情绪与关系温度

更低成本:让跨地域沟通与群体沟通不再受制于时间与费用

更可靠:在弱网与断线情况下保持一致体验,尽量减少丢失与重复

- 1.2 Business Case
  - 项目目标

开发一款高性能、高可用的普适即时通讯系统,采用 MoSCoW 方法进行功能优先级管理:

Must Have (MVP 核心)

账号体系(手机号/邮箱注册登录)

联系人体系(添加/删除/备注/黑名单)

私聊/群聊

文本/图片/语音/表情消息

文件发送(小文件)

Server-ACK 机制

离线消息拉取

桌面客户端

Should Have

搜索(按关键词/联系人/群组)

已读/未读与未读计数

置顶会话与消息引用回复

消息撤回与转发

Could Have

消息收藏

群公告与群管理(入群方式、禁言、管理员)

断网/弱网重连动画

陌生人消息请求与反骚扰策略(开关、频控、举报)

Won't Have (本轮不做)

视频/语音通话

多端同步

实时协作文档

- 预期收益

用户价值(大众用户):随时联系、即时回应,跨地域沟通成本显著降低,社交关系维护更自然、更稳定

用户价值(群体关系):群聊信息更清晰,重要信息更不易错过,降低重复确认与误解成本

技术价值:提供零丢失、低延迟(≤200ms)、稳定可靠的实时通讯服务

学习价值:完整展示 IM 系统的工业级架构设计,包含 NFR 管理、协议设计、可靠性机制与性能压测方法

- 约束与假设

维度	约束条件

- 非功能性需求 (NFR)

维度	指标要求	验证方法

- 愿景

2.1 问题陈述

1. 问题一：即时联系成为刚需,基础体验要求更高

要素	描述
问题	手机普及后,“随时联系、即时回应”成为大众沟通的基本预期,但在弱网、断线、设备切换与高频群聊等场景下,消息到达与阅读状态不确定、重要信息易错过、回看与检索成本高等问题依然普遍存在。
影响群体	大众用户(朋友、家人、异地恋、同学等社交关系)、需要跨地域保持联系的人群。
后果	错过关键信息、反复确认造成沟通疲劳,并在高频互动中引发误解与情绪摩擦,抬高关系维护成本。
解决方案	ShirohaChat 聚焦“及时触达与稳定可靠”:提供 Server-ACK、离线补发与断线重连,并配套清晰的未读计数、搜索与隐私/反骚扰设置,让沟通更省心。

2. 问题二：现有方案的技术局限

要素	描述
问题	主流闭源 IM 系统难以进行架构研究与二次开发，开源方案多为 Demo 级别，缺乏工业级设计。
影响群体	软件工程学习者、系统架构研究者。
后果	理论与实践脱节，无法深入理解 WebSocket、消息可靠性、分布式状态管理等核心技术。
解决方案	ShirohaChat 开源完整架构设计（含 NFR、协议、风险管理），可作为学习与扩展基础。

2.2 涉众与用户

1. 涉众分析

涉众类型	代表	核心诉求	优先级
项目团队	开发组/测试组	掌握工业级 IM 架构设计与实现能力	高
目标用户	大众用户	稳定、易用的日常即时通讯工具	高
技术评审者	导师/面试官	系统展示专业性（架构、性能、文档）	高
潜在扩展者	开源社区开发者	清晰的代码结构与接口文档	中

2. 用户画像(典型大众场景)

2.1. 小周 - 异地恋用户

- 场景:日常高频私聊,需要在不同时间与地点保持“随时联系、即时回应”
- 痛点:

  弱网或断线时消息状态不明确,容易产生“是否收到”的焦虑
  夜间或工作时段需要更细粒度的通知与免打扰控制
  希望表达更自然(语音、图片、表情),但不希望操作复杂
  期望:消息到达更确定、通知可控、表达顺畅且操作简单

1.1. 阿慧 - 家庭群组织者

- 场景:家庭群日常沟通与信息转达(聚餐时间、出行信息、照片分享等)
- 痛点:

  群聊刷屏后重要信息容易被淹没,成员反复询问
  老人不熟悉复杂操作,需要更直观的未读与提示
  图片与文件较多时,回看与查找成本高
- 期望:群聊信息更清晰、老人也能轻松使用、历史内容更好找

1.1. 小林 - 与陌生人沟通的用户

- 场景:二手交易、兴趣社交等场景下与陌生人短期沟通
- 痛点:

  易收到骚扰信息,需要明确的消息请求与拦截机制
  希望能快速建立联系,同时保护隐私

沟通结束后希望一键清理并控制对方可见范围

- 期望:更强的反骚扰与隐私设置、陌生人沟通更安全可控

2.3 关键涉众和用户的需要

涉众/用户	核心需求	系统响应
大众用户	低延迟、消息可靠到达、离线补发、通知可控、隐私与反骚扰	Server-ACK + 离线补发 + 未读计数/搜索 + 黑名单/消息请求
开发者	模块化架构、清晰接口文档、易于扩展	API 文档 + 分层设计 + 代码规范
评审者	展示系统专业性（性能指标、架构图、风险管理）	NFR 量化 + 压测报告 + UML 建模
运维者	一键部署、监控日志、故障恢复	容器化部署 + 日志系统

2.4 产品概述

1. 1. 产品定位陈述

For 大众用户(朋友、家人、异地恋、同学与陌生人沟通场景)
Who 在跨地域与高频社交沟通中需要“随时联系、即时回应”,并希望消息更可靠、通知更可控、隐私更安心
The ShirohaChat
Is a 面向大众的桌面端即时通讯工具
That 提供私聊/群聊、多媒体消息、未读与搜索、黑名单与消息请求等能力,并以 Server-ACK 与离线补发保证消息可靠送达
Unlike 微信(功能堆叠、入口复杂)、QQ(界面复杂、广告干扰)、Discord(国内访问难)
Our product 以“简洁、可靠、可控”为优先,同时保证消息可靠性≥99.99%、端到端延迟≤200ms,提供更聚焦的桌面端沟通体验

2. 2. 完整的产品概述

ShirohaChat 是一款面向大众的即时通讯系统**(**桌面端**)**,采用前后端分离的 C/S 架构:

前端:桌面客户端,提供联系人与会话列表,支持文本/图片/语音/表情与文件消息展示,并提供搜索与未读管理
后端:实时通讯服务,实现 Server-ACK、离线消息队列、在线状态与重连机制
数据层:持久化存储用户/群组/消息数据,缓存在线状态与离线消息队列
核心特性:

- 日常沟通:私聊/群聊、多媒体消息、引用回复与转发
- 信息管理:未读计数、消息搜索、置顶会话与收藏
- 隐私与安全:黑名单、陌生人消息请求、基础反骚扰策略
- 消息可靠性:Server-ACK 机制、离线补发、消息零丢失(≥99.99%)
- 高性能:端到端延迟≤200ms,单节点支持≥5000 并发连接

- 用况建模

- 术语表

用户（User）	使用聊天系统的注册使用者
好友（Friend）	与用户建立双向连接关系的其他用户
群聊（Group Chat）	由多个用户组成的消息会话
消息（Message）	文本、图片、文件或表情形式的数据

- ShirohaChat 的主要用况
- 主要用况的描述

1.1. （1）即时通讯（聊天）

- 简要描述

该用况描述用户选择会话、发送文本消息、查看实时反馈以及接收对方消息的完整交互过程

- 用况图

- 前置条件：

系统进入 Chat 用况前必须满足：

1、用户已登录系统。

2、用户的网络连接状态正常。

3、用户的联系人列表中至少存在一个好友或群组。

基本流（Basic Flow）

{进入会话}

1、用户在主界面的会话列表中，选择一个联系人或群组进行点击。

2、系统打开该会话的聊天窗口，并显示之前的历史聊天记录

{编辑与发送}}

3、用户在输入框中输入文本消息。

4、用户点击“发送”按钮（或按下回车键）。

{接收消息}

5、系统在聊天窗口中实时显示对方发来的新消息。

6、系统自动滚动聊天视图以展示最新内容

3.4.2 备选流 (Alternative Flows)

A1：收到来自其他会话的消息 *在基本流执行期间，若收到非当前会话的消息：*

1. 系统在左侧会话列表中，将新消息来源的会话置顶。

2. 系统在该会话项上显示未读消息提示（如红点或数字）。

3. 用户继续在当前窗口操作，不受干扰。

A2：查看历史消息 *在基本流步骤 2 之后的任意时刻：*

1. 用户向上滚动聊天窗口。

2. 系统加载并显示更早之前的聊天记录。

3. 用户停止滚动，用况恢复到当前步骤。

3.4.3 异常流 (Exception Flows)

E1：发送空消息 *在基本流步骤 4，若用户输入内容为空或仅含空格：*

1. 系统检测到内容无效。

2. 系统保持发送按钮不可点击，或弹出提示“无法发送空内容”。

3. 用况回到基本流步骤 3。

E2：发送失败（网络异常） *在基本流步骤* 5 *之后，若系统无法连接服务器：*

1. 系统在超时或检测到断开后，将该消息的状态标识更新为“发送失败”（如红色感叹号）。

2. 用户点击该“失败标识”。
3. 系统尝试重新发送。
    - E2a 重发成功：用况回到基本流步骤 6。
    - E2b 重发失败：系统再次显示“失败标识”，等待用户下一次操作。

后置条件 (Post-conditions):

- 成功：消息内容被系统存储，且界面显示“已送达”。
- 失败：消息内容被本地保存（不丢失），界面显示“发送失败”。

公共流（Common Flows）

UC-01：注册账户 (Register Account)

1. **简要描述** 该用况描述用户首次使用系统时，填写身份信息并创建新账户的全过程。
2. **前置条件**
    - 用户尚未拥有本系统账户。
    - 系统服务处于可用状态。
3. **基本流**
    1. **用户**打开应用，点击“注册”按钮。
    2. **系统**显示注册界面，包含账号、密码、确认密码、昵称输入框。
    3. **用户**填写上述信息并提交。
    4. **系统**提示“注册成功”。
    5. **用户**选择跳转至登录界面。
4. **备选流 / 异常流**
    - **A1：账号已存在**

    - 系统提示“该账号已被注册”，并停留在注册界面。
- **A2：密码不一致**
    - 系统提示“两次输入的密码不一致”。
- **A3：输入不合规**
    - 系统提示具体的格式错误（如“密码需包含字母与数字”、“必填项不能为空”）。
- **A5：服务不可用**
    - 系统提示“注册服务暂时不可用，请稍后重试”。

UC-02：登录系统 (Login)

1. **简要描述** 该用况描述用户使用已有账户进入系统主界面的过程。
2. **前置条件**
    - 用户已完成注册。
    - 用户的网络连接正常。
3. **基本流**
    1. **用户**启动应用进入登录界面。
    2. **用户**输入账号与密码并点击“登录”。
    3. **系统**检查输入格式是否有效。
    4. **系统**跳转到主界面，显示会话列表。

- 需求分析

- 4.1 健壮性分析 (Robustness Analysis)

健壮性分析通过区分三类对象（边界、控制、实体）来验证用况逻辑的完整性。针对本次迭代的核心用况 **UC-01 即时通讯**，我们识别出以下关键分析类：

4.1.1 识别分析类

1. 边界对象 (Boundary Objects)

- 职责：系统与外部参与者交互的接口。
- `ChatWindow` (聊天窗口)：显示消息列表、接收用户输入、展示发送状态（转圈/对勾）。
- `SessionList` (会话列表)：显示好友/群组列表及未读红点。

2. 控制对象 (Control Objects)

- 职责：协调业务逻辑，连接边界与实体。
- `ChatController` (聊天控制器)：核心调度者。负责验证消息、创建消息实体、调用网络服务、更新 UI 状态。
- `NetworkService` (网络服务)：负责底层通信（发送数据包、监听接收）。

3. 实体对象 (Entity Objects)

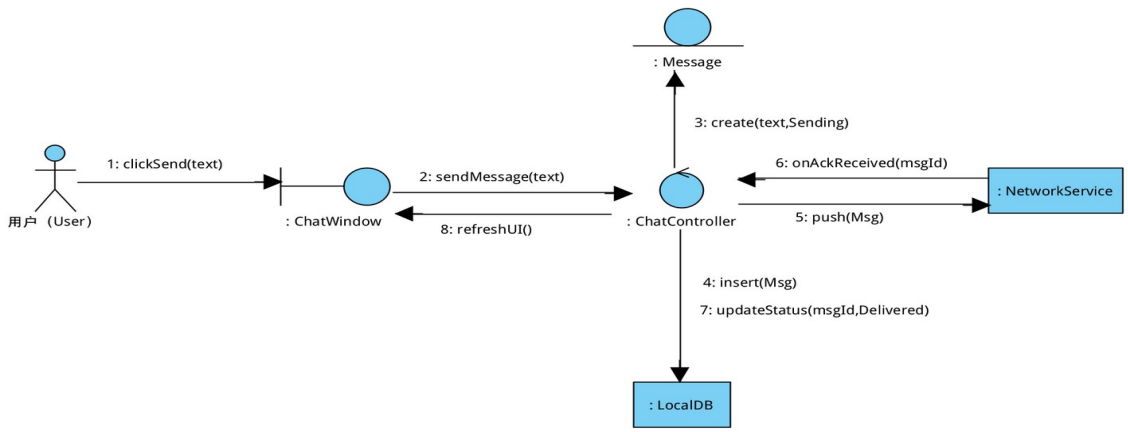
- 职责：存储系统需要长久保存的数据。
- `Message` (消息)：包含内容、发送者、时间戳、状态。
- `User` (用户)：包含用户 ID、昵称、头像。
- `Session` (会话)：维护当前聊天的上下文（是跟谁聊）。

4.2 交互建模：通信图

根据 UML 2.2 第 9 章，通信图强调**对象之间的链接关系**。这对于发现类图中的“关联”至关重要。

我们选取 UC-01 的基本流（发送消息）进行建模。

图 4-1 发送消息通信图:



4.2.2 通信图分析与链接识别

通过 4 图，我们明确了对象在空间上的结构关系（即“谁认识谁”）：

1. **UI 与 控制器**：`ChatWindow` 向 `ChatController` 转发用户操作（`2: sendMessage`），并在最后接收控制器的刷新通知（`8: refreshUI`）.界面层必须持有控制器的引用，以便触发核心业务逻辑，通知其更新消息状态（如将“转圈”动画改为“对勾”）。

2. **控制器 与 实体**：`ChatController` 创建并操作 `Message`（3: create），说明控制器管理着消息实体的生命周期，控制层依赖于基础设施层的数据访问能力。

3. **控制器 与 网络**：`ChatController` 调用底层网络发送数据（`5: push`），网络层随后触发异步回调（`6: onAckReceived`），控制器依赖网络服务与 ShirohaChat 服务端进行通信。

4..2.3 从通信图导出分析类图

根据通信图中识别出的**链接 (Links)** 和**消息 (Messages)**，我们可以直接转化为类图中的**关联 (Associations)** 和**操作 (Operations)**。

- **链接 -> 关联**：通信图中两个对象之间有连线，类图中这两个类之间就有关联关系。
- **消息 -> 方法**：对象 A 向 对象 B 发送消息 `doSomething()`，则 类 B 中必须定义方法 `doSomething()`

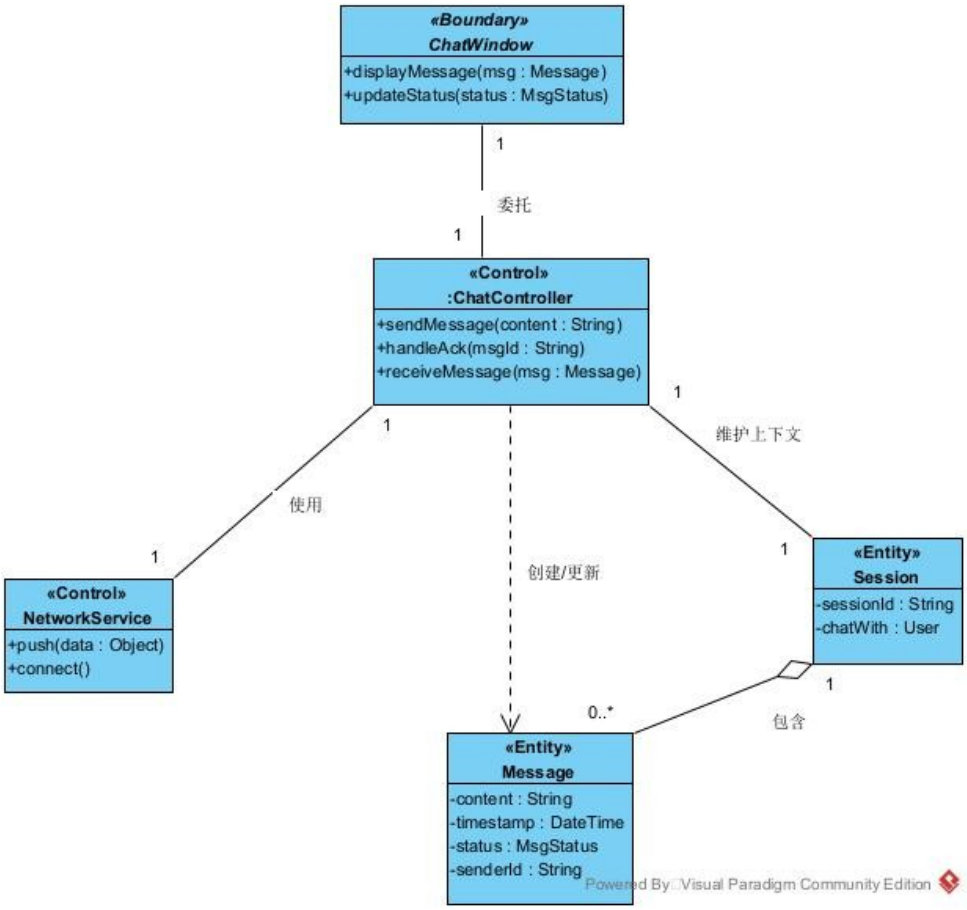


图 4-2 类图

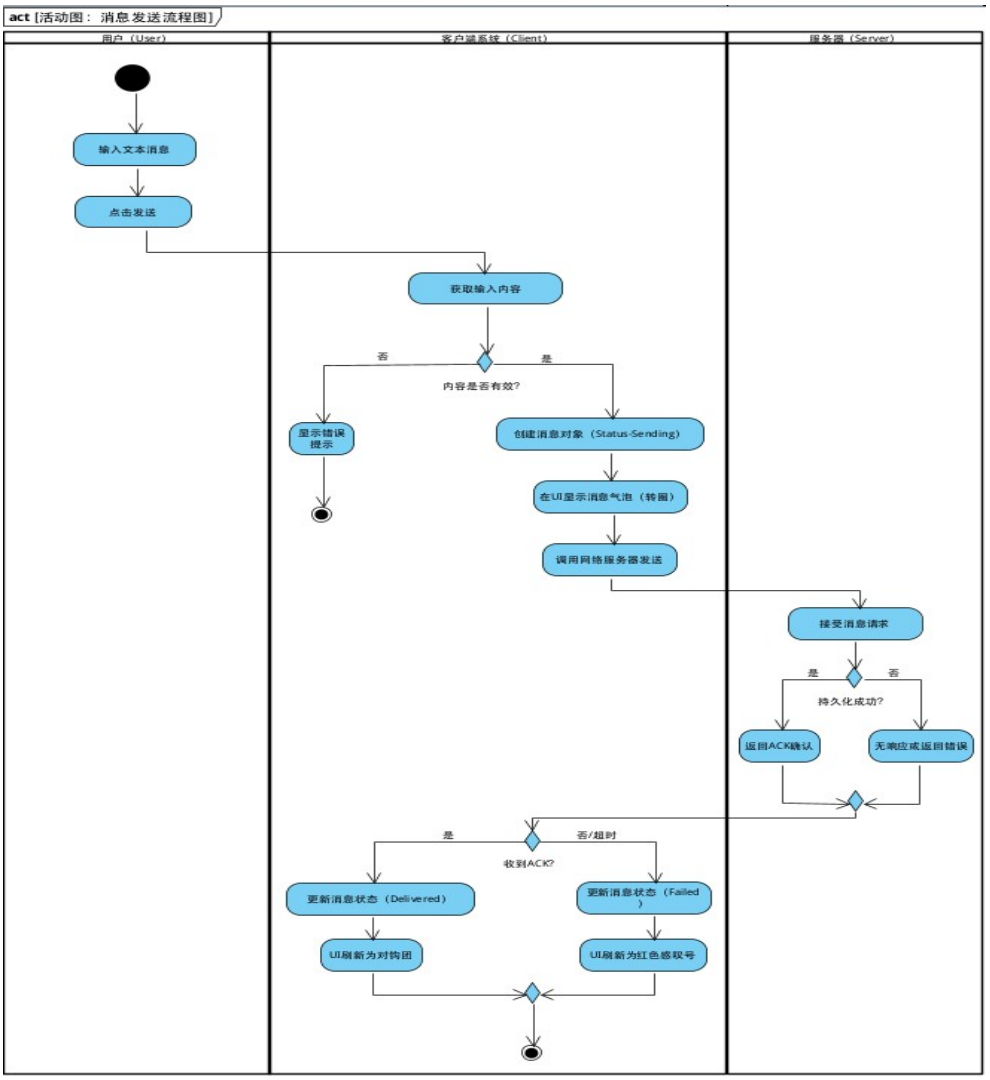
4.2.4 类职责说明

- **ChatWindow**: 负责界面的渲染。它不包含业务逻辑，只负责将用户的点击转发给 `ChatController`。
- **ChatController**: 系统的核心。它隔离了界面和网络，确保界面不知道网络是如何实现的（符合黑盒原则）。它持有 `NetworkService` 的句柄来发送数据。
- **Message**: 纯数据类（Data Object）。包含第一次迭代所需的属性，如 `content` (内容) 和 `status` (用于 UI 显示转圈或对勾)。

4.3 交互建模

4.3.1 活动图

图 4-3 发送消息活动图 该图展示了消息从用户输入到最终状态更新的完整业务逻辑流



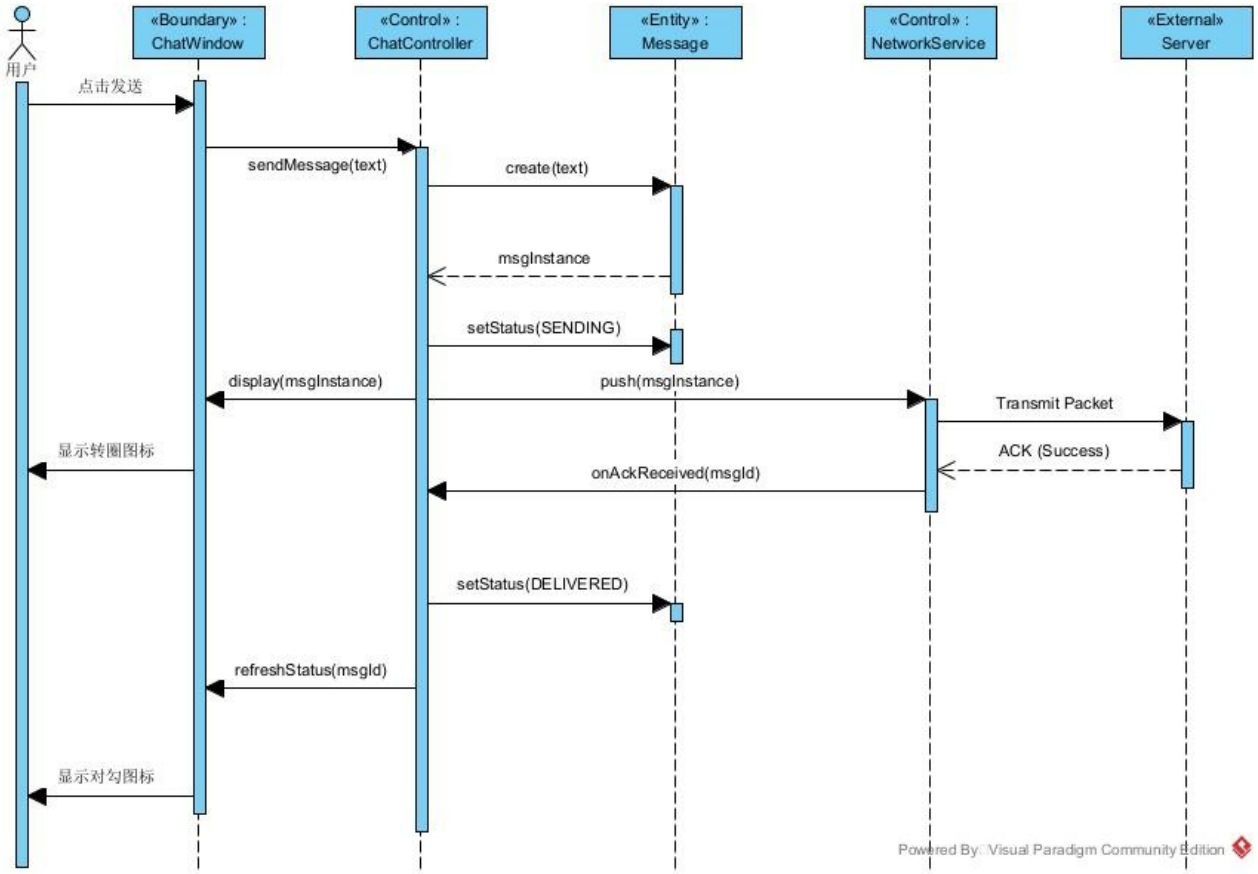
活动图说明：

- **泳道 (Swimlanes)**：将活动划分为“用户”、“客户端系统”、“服务器”三个责任区，明确了职责边界。
- **决策节点 (Decision Nodes)**：图中包含两个关键判断：
    1. **前置校验**：在发送前拦截非法内容（空消息）。
    2. **后置确认**：根据是否收到 ACK 决定消息最终状态，这是保证数据一致性的逻辑核心。

4.3.2 顺序图

图 4-4 消息发送与 ACK 机制顺序图 该图依据 UML 2.2 规范，展示了边界对象、控制对象与实体对象之间的时间序列交互。

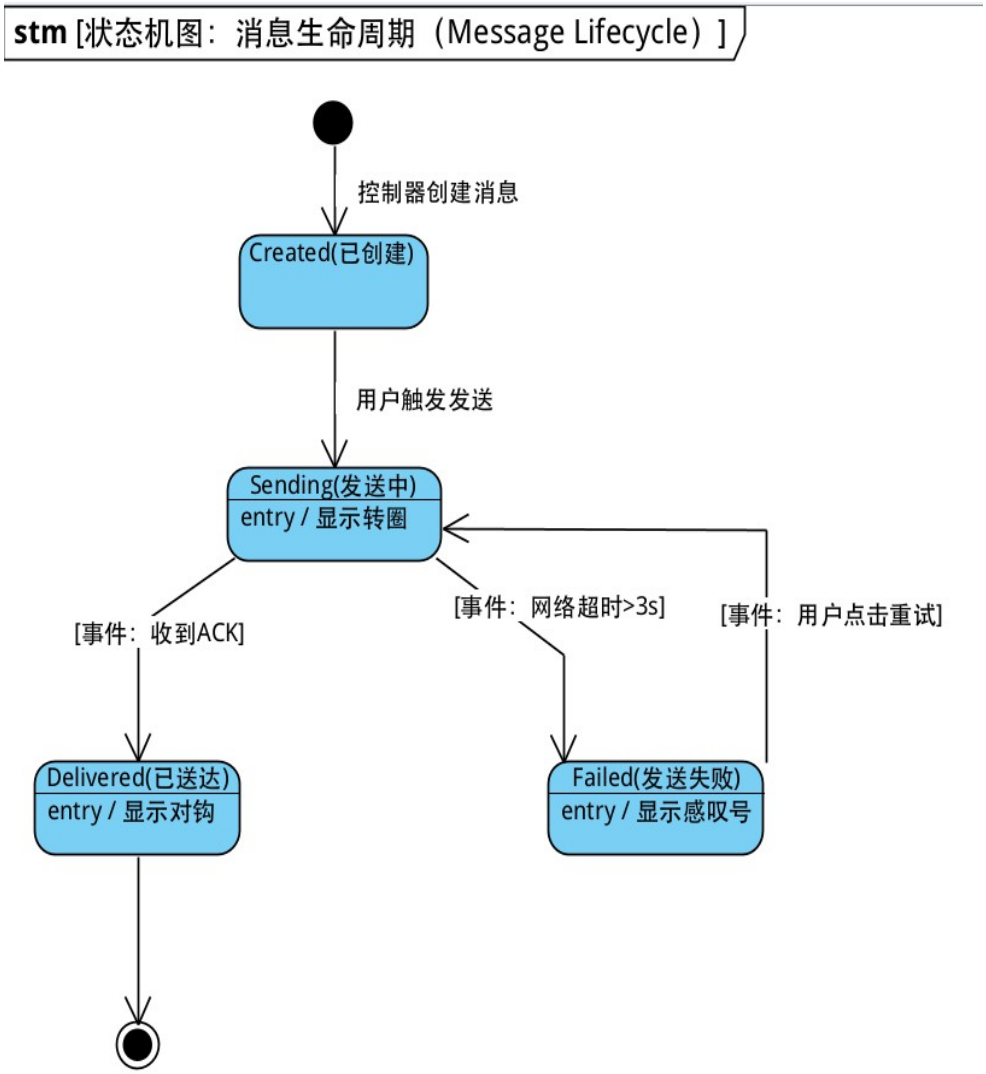
顺序图说明：



- **生命线 (Lifelines)**：清晰列出了参与交互的所有对象。
- **激活条 (Activation Bars)**：细长的矩形表示对象处于活动（处理）状态的时间段。
- **消息类型**：
  - **同步消息**（实心箭头）：如 `sendMessage()`，表示调用后等待返回。
  - **异步消息**（开箭头）：如 ACK 回执，表示网络层的异步通知。
- **对象创建**：消息 3 (`create`) 展示了 `Message` 实体是在发送过程中被动态创建的。

4.3.3 状态机图

图 4-5 消息实体状态机图 根据项目“高可靠性”的需求，`Message` 对象不仅是数据容器，还是一个具有复杂生命周期的状态机。



状态机图说明：

- **状态 (States)**：
  - `Sending`：中间瞬态。进入此状态时（entry action）会启动超时计时器。
  - `Delivered`：最终成功状态。
  - `Failed`：异常状态。此状态允许转换回 `Sending` 状态（即重发机制）。
- **转换 (Transitions)**：

- `[事件: 网络超时 > 3s]`：这是一个**监护条件 (Guard Condition)**，定义了从发送中变为失败的触发逻辑。
- `Failed --> Sending`：这个闭环路径完美解释了需求中的“重试机制”。

4.4 需求分析小结

通过本章的分析工作，我们完成了从外部需求到内部逻辑的映射：

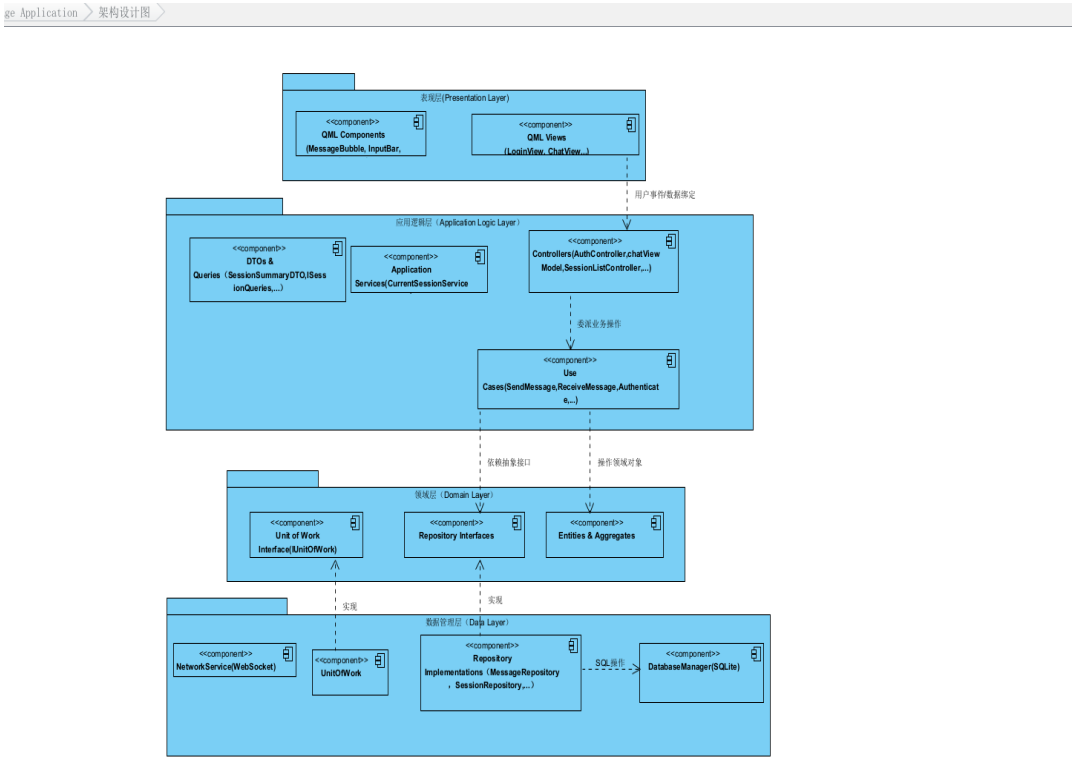
1. **健壮性分析** 识别了 `ChatController` 和 `Message` 等关键类。
2. **通信图** 确定了这些类之间的静态链接关系。
3. **活动图** 梳理了业务逻辑的分支判断。
4. **顺序图** 验证了对象协作的时序正确性。
5. **状态机图** 确保了核心对象生命周期的完整性。

- 架构设计

本章在需求分析模型的基础上，定义 ShirohaChat 的高层系统架构。我们采用 **C/S（客户端-服务器）架构** 结合 **分层模式 (Layered Pattern)**，以满足“端到端延迟 ≤ 200ms”和“消息零丢失”的非功能性需求

5.1 架构概述

ShirohaChat 采用经典的四层面向对象架构,从上到下依次为:表现层、应用逻辑层、领域层和数据管理层。系统整体采用 C/S(客户端-服务器)架构,客户端与服务端通过 WebSocket 协议进行实时双向通信。



架构设计决策说明

1、依赖方向: 始终从上层向下层依赖,下层不依赖上层。领域层定义仓库接口,数据管理层提供实现——这是典型的依赖倒置原则(DIP)。

2、组合根: 客户端使用 AppCoordinator 作为纯 C++ 组合根,在 main() 中统一创建所有基础设施、用例和控制器实例,并完成依赖注入。AppCoordinator 本身不暴露给 QML。

3、控制器作为用况控制器: 每个 Controller 对应一组相关的用况,是 QML 视图与业

务逻辑之间的中介。Controllers 使用 QML_SINGLETON 宏注册为 QML 单例,由 QML 引擎直接访问。

5.2 客户端架构

5.2.1 表现层

表现层完全由 QML 实现,职责是用户界面渲染与用户交互捕获。表现层不包含任何业务逻辑,所有用户操作都通过属性绑定和方法调用委托给应用逻辑层的 Controller。

视图组件文件	职责	绑定的 **Controller**
UI/Views/LoginView.qml	登录/注册界面	AuthController
UI/Views/SessionSidebar.qml	会话列表展示	SessionListController, AuthController
UI/Views/ChatView.qml	聊天消息展示与发送	ChatViewModel
UI/Views/GroupInfoView.qml	群组信息与成员管理	GroupController
UI/Views/AddFriendDialog.qml	添加好友弹窗	ContactController
UI/Views/FriendRequestsDialog.qml	好友请求处理	ContactController
UI/Views/CreateGroupDialog.qml	创建群组弹窗	GroupController

5.2.2 应用逻辑层

应用逻辑层由 Controllers(用况控制器)和 Use Cases(用例对象)组成,是系统行为逻辑的核心所在。

Controllers: 面向表现层的适配器,持有 QML_SINGLETON 标记,将 QML 的用户事件转化为 UseCase 调用,并将结果通过 Qt 属性通知机制反馈给 UI。

Controller	关联的 **Use Cases**	职责概述
AuthController	AuthenticateUseCase	登录/注册/登出的 UI 协调
ChatViewModel	SendMessageUseCase, ReceiveMessageUseCase	当前聊天窗口的消息展示与发送
SessionListController	SwitchSessionUseCase, SessionSyncUseCase	会话列表维护与切换
ContactController	FriendRequestUseCase	好友关系管理
GroupController	GroupManagementUseCase	群组创建与成员管理

5.2.3 领域层

领域层是系统的业务知识核心,包含所有实体类、值对象、聚合根以及仓库接口定义。领域层不依赖任何框架或基础设施代码(仅使用 Qt 的基础类型如 QString、QDateTime)。

实体/聚合	文件	职责	关键行为方法
Message	shared/domain/message.h	单条消息, 维护状态机	markDelivered(), markFailed(), isValid()
Session	shared/domain/session.h	会话聚合根, 拥有摘要与未读计数	processIncomingMessage(), markRead(), updateMetadata()
Group	shared/domain/group.h	群组聚合根, 管理成员集合	addMember(), removeMember(), canAddMember()
GroupMember	shared/domain/group.h	群组成员值对象	—
User	shared/domain/user.h	用户实体, 包含昵称验证逻辑	changeNickname(), isValid()
Contact	shared/domain/contact.h	联系人关系实体, 维护状态机	acceptRequest(), block(), unblock()
FriendRequest	shared/domain/friend_request.h	好友请求实体, 包含授权校验	accept(), reject(), cancel()

5.2.4 数据管理层

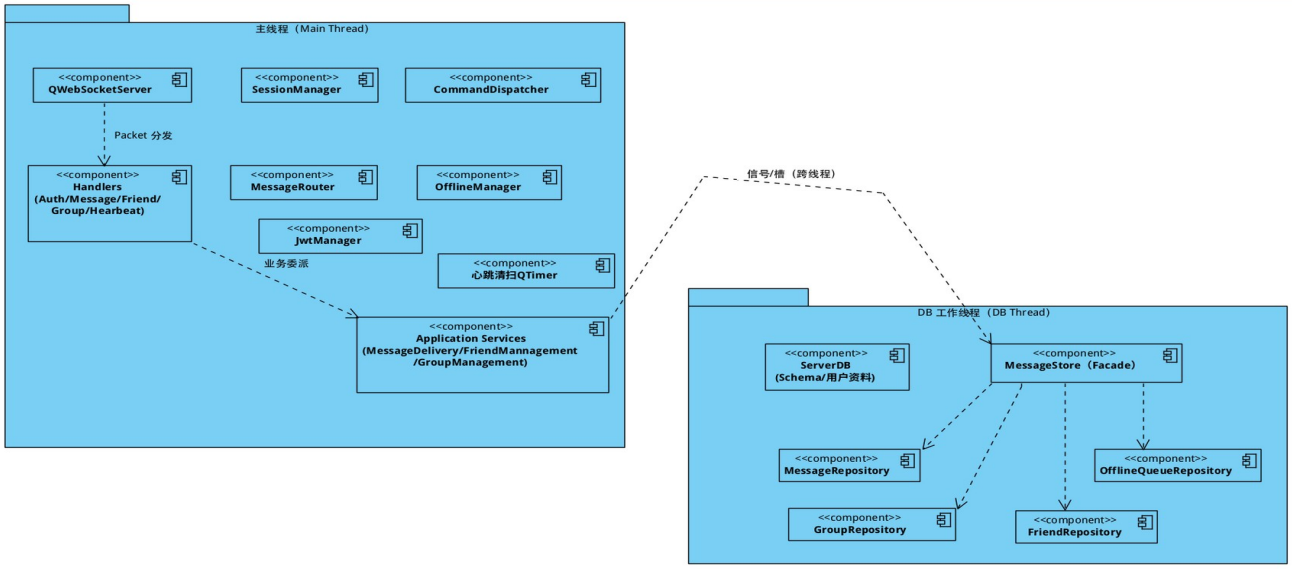
数据管理层负责领域对象的持久化和外部通信。

仓库实现	实现的接口	存储介质
MessageRepository	IMessageRepository	SQLite (messages 表)
SessionRepository	ISessionRepository	SQLite (sessions 表)
GroupRepository	IGroupRepository	SQLite (groups + group_members 表)
UserRepository	IUserRepository	SQLite (users 表)
FriendRepository	IContactRepository + IFriendRequestRepository	SQLite (contacts + friend_requests 表)
AuthStateRepository	IAuthStateRepository	SQLite (auth_state 表)

5.3 服务端架构

服务端同样遵循四层架构,但由于没有图形界面,其"表现层"体现为 WebSocket 接口层。

5.3.1 线程模型



关键设计:

1、主线程负责所有 WebSocket I/O 和业务调度,保证 QWebSocket 操作的线程安全

2、DB 工作线程持有 ServerDB 和 MessageStore(门面模式),通过 Qt 信号/槽的 QueuedConnection 与主线程异步通信

3、MessageStore 封装了 4 个具体仓库,对主线程暴露统一的存储接口

5.3.2 消息投递机制

私聊消息流:

发送方客户端发送 SendMessage 包

MessageHandler 校验参数后交给 MessageDeliveryService

幂等性检查(防止客户端重试导致重复处理)

立即向发送方返回 SendMessageAck

消息持久化到 MessageRepository

构造 ReceiveMessage 包,通过 OfflineManager → MessageRouter 投递

若接收方在线:直接推送;若离线:存入 OfflineQueueRepository

接收方上线后触发 deliverOnLogin() 离线补发

群聊消息流:

MessageDeliveryService 异步加载群成员列表

验证发送者是否为群成员

向发送者以外的所有成员做 fan-out 投递

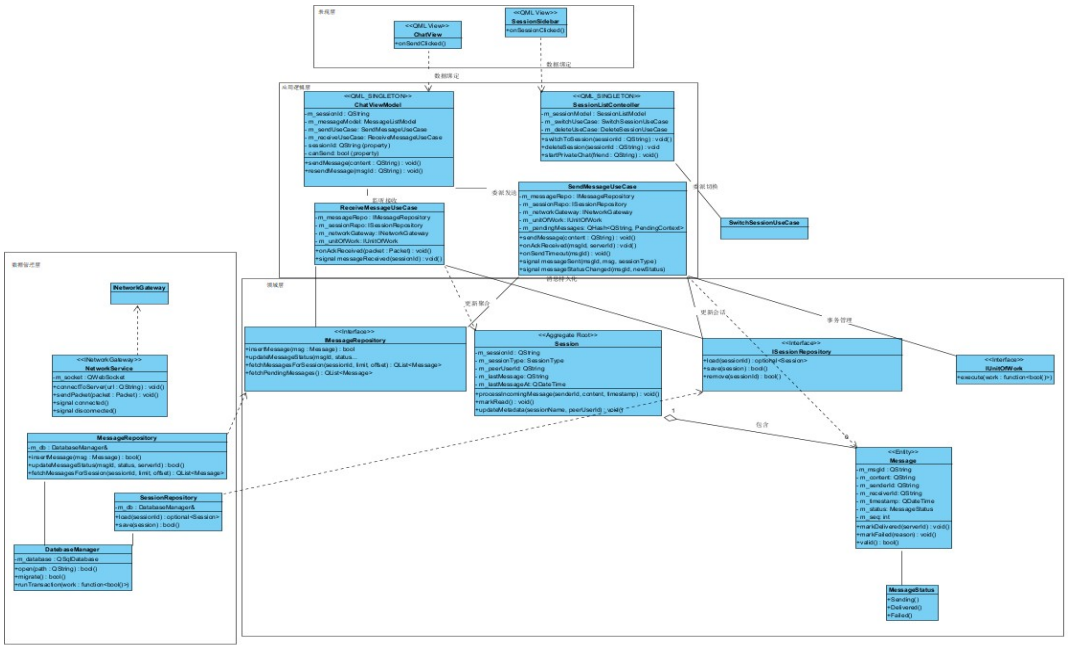
每个成员走相同的在线/离线投递逻辑

- 详细设计

6.1 设计类图

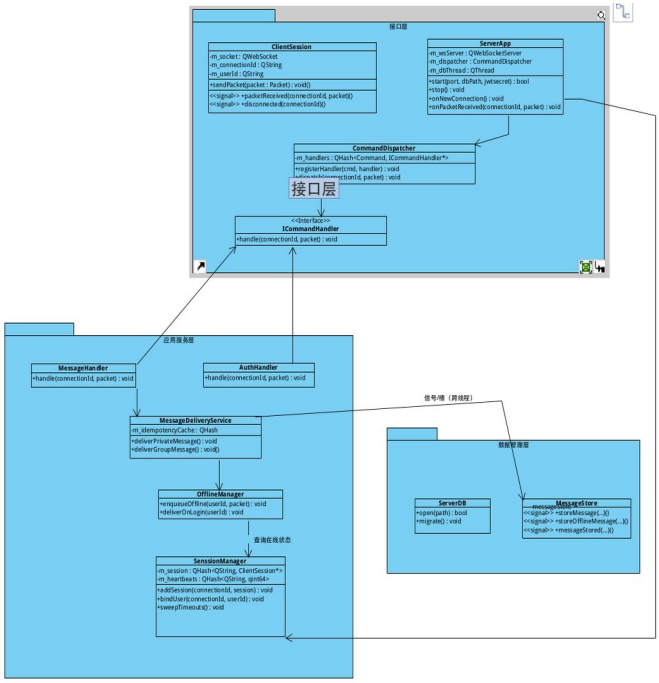
基于第 4 章的分析类图和第 5 章的架构设计,将分析阶段的类细化为可实现的设计类。以下是第一次迭代(即时通讯核心流)涉及的关键设计类。

6.1.1 客户端核心类图



6.1.2 服务端核心类图

6.2 设计模式应用



本系统在实现过程中有意识地应用了多种面向对象设计模式,以下是主要模式的映射:

1、仓库模式 (Repository) IMessageRepository / MessageRepository 等 将领域层与数据访问细节解耦,领域层只依赖接口
2、观察者模式 (Observer) Qt signals/slots 贯穿全系统 网络层到用例层、用例层到控制器、控制器到视图的事件通知
3、单例模式 (Singleton) AppCoordinator::instance()、各 Controller 的 QML_SINGLETON保证全局唯一实例,QML 引擎可直接访问

6.3 数据库设计

6.3.1 客户端数据库表

表名	说明	主要字段
auth_state	认证状态 (单行)	token, userId, nickname, serverUrl
users	用户信息缓存	userId (PK), nickname, lastSeenAt
sessions	会话列表	sessionId (PK), sessionType, ownerUserId, sessionName, peerUserId, lastMessageContent, lastMessageAt, unreadCount
messages	消息记录	msgId (PK), content, senderId, sessionId (FK→sessions), timestamp, status, serverId
contacts	联系人关系	userId + friendUserId (PK), status, createdAt
friend_requests	好友请求	requestId (PK), fromUserId, toUserId, message, status, createdAt, handledAt
groups	群组信	groupId (PK), groupName, creatorUserId, createdAt

表名	说明	主要字段
	息	
group_members	群组成员	groupId + userId (PK), role, joinedAt
pending_acks	待确认消息 (发送中)	msgId (PK)

6.3.2 服务端数据库表

表名	说明	主要字段
users	注册用户	userId (PK), nickname, passwordHash, createdAt
messages	消息持久化	serverId (PK), msgId, senderId, sessionId, sessionType, content, timestamp
offline_queue	离线消息队列	id (PK), userId, packetJson, enqueuedAt
groups	群组信息	groupId (PK), groupName, creatorUserId, createdAt
group_members	群组成员	groupId + userId (PK), role, joinedAt
friends	好友关系	userId + friendUserId (PK), createdAt

表名	说明	主要字段
friend_requests	好友请求	requestId (PK), fromUserId, toUserId, message, status, createdAt

后记

ShirohaChat 从需求捕获到系统实现,经历了完整的软件工程开发过程。在这个过程中,我们深刻体会到几点:

一是用况建模的关键在于始终以用户目标为导向。开发初期容易犯的错误是将系统功能列表直接等同于用况,导致模型退化为功能分解。在反复推敲后,我们将用况数量从十几个精简为 6 个用户目标级别的用况,每个用况清晰地回答"用户想要达成什么"。

二是四层架构的分层纪律需要持续维护。在实际编码过程中,为了开发效率,很容易出现跨层调用的情况(比如控制器直接操作数据库)。我们通过定义领域层的仓库接口、引入依赖注入和组合根模式,在架构层面保证了分层的严格性。

三是领域对象应该拥有行为。贫血模型(只有 getter/setter 的纯数据类)是面向对象设计的反模式。在本项目中,Message 的状态机转换、Group 的成员关系不变式、FriendRequest 的参与者授权检查,都体现了"让数据和行为待在一起"的 OOP 核心原则。

最后,这个项目让我们将课程中学习的需求管理、UML 建模、面向对象分析与设计等方法论,在一个真实的 IM 系统中得到了完整的实践。

参考文献

Dean Leffingwell, Don Widrig. Managing Software Requirements: A Use Case Approach (Second Edition). Addison-Wesley, 2003.
Daryl Kulak, Eamonn Guiney. Use Cases: Requirements in Context (Second Edition). Addison-Wesley, 2004.
Michael Blaha, James Rumbaugh. Object-Oriented Modeling and Design with UML (Second Edition). Pearson, 2005.
Simon Bennett, Steve McRobb, Ray Farmer. Object-Oriented Systems Analysis and Design Using UML (Fourth Edition). McGraw-Hill, 2010.
Qt Group. Qt 6.5 Documentation. https://doc.qt.io/qt-6/
Martin Fowler. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.
Eric Evans. Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-

Wesley, 2003.